OSTEP 精简版

Operating Systems: Three Easy Pieces

后端面试速查笔记

虚拟化 · 并发 · 持久化

目录

进程

- 机器状态
- 进程 API
- 进程创建过程
- 进程状态
- 进程控制块

进程 API（UNIX）

- fork()
- wait()
- exec()
- 为什么要分离 fork 和 exec
- shell 的工作原理
- pipe()
- 其他 API

受限直接执行

- 用户模式与内核模式
- 系统调用与 trap
- trap 表
- 进程切换
- 上下文切换
- LDE 协议总结

调度：基础策略

- 评价指标
- 简单算法速览

多级反馈队列（MLFQ）

- 基本结构
- 五条规则
- 为什么这样设计
- 实际系统中的 MLFQ
- 参数调优

多处理器调度

- 缓存与一致性
- 缓存亲和性
- 两种调度方案
- Linux 调度器

地址空间

- 从物理内存到虚拟化

地址空间的布局
虚拟内存的三个目标
内存 API
栈内存 vs 堆内存
核心 API
常见错误
底层机制
分页
页表
页表项（PTE）
页表太大的问题
TLB
工作流程
硬件管理 vs 软件管理
TLB 内容
上下文切换时怎么办
替换策略
更小的页表
方案一：更大的页
方案二：分页与分段结合
方案三：多级页表
方案四：反向页表
超越物理内存：机制
交换空间
存在位
页错误处理
硬件控制流
水位线机制
页替换策略
评价指标
几种策略
LRU 的近似实现——时钟算法
工作负载与策略选择
抖动
并发与线程
线程
为什么需要线程
共享数据的问题
核心概念

线程 API（POSIX）

- 创建线程
- 等待线程
- 互斥锁
- 条件变量
- 编译

锁

- 关中断
- 自旋锁
- 自旋的问题
- 基于队列的休眠锁
- Linux futex
- 两阶段锁

基于锁的并发数据结构

- 并发计数器
- 并发链表
- 并发队列（Michael-Scott 队列）
- 并发哈希表

条件变量

- 基本操作
- 生产者/消费者（有界缓冲区）
- 覆盖条件

信号量

- 基本操作
- 当锁用
- 当条件变量用
- 生产者/消费者
- 读者-写者锁
- 哲学家就餐
- 用锁和条件变量实现信号量

常见并发 Bug

- 非死锁 Bug
- 死锁

基于事件的并发

- 事件循环
- select()/poll()
- 阻塞操作的问题
- 状态管理
- 多核上的局限

I/O 设备
- 系统架构
- 设备接口
- 与设备交互的方式
- 轮询 vs 中断
- DMA
- 设备驱动

磁盘驱动器
- 物理结构
- I/O 时间
- 顺序 vs 随机
- 磁盘调度
- 其他优化

文件与目录
- 文件
- 目录
- 文件操作 API
- 硬链接与软链接
- mount

文件系统实现
- 磁盘布局
- inode
- 目录
- 空闲空间管理
- 读文件的路径
- 写文件的路径
- 缓存

崩溃一致性与日志
- fsck：事后修复
- 日志（Journaling）
- 其他方案

第一部分：虚拟化

进程

进程就是运行中的程序。程序本身是静态的，存在磁盘上；操作系统把它跑起来，就成了进程。

机器状态

进程能读写的所有东西，构成它的机器状态（machine state）：

- 地址空间：进程能访问的内存
- 寄存器：程序计数器（PC）记录下一条指令地址；栈指针（SP）和帧指针（FP）管理函数调用栈
- I/O 信息：进程打开的文件列表（文件描述符）

进程 API

操作系统提供的进程操作接口：

- 创建（create）：生成新进程
- 销毁（destroy）：强制杀死进程
- 等待（wait）：等某个进程结束
- 状态查询（status）：获取进程信息

进程创建过程

1. 把程序代码和静态数据加载到内存（现代系统用懒加载，按需加页）
2. 分配栈空间，初始化 `argc`/`argv`
3. 分配堆空间
4. 做其他初始化（比如打开 fd 0/1/2 对应 stdin/stdout/stderr）
5. 跳到 `main()` 入口，OS 把 CPU 控制权交给进程

进程状态

三种基本状态：

- 运行（Running）：正在 CPU 上执行指令
- 就绪（Ready）：可以运行但 OS 还没调度到它
- 阻塞（Blocked）：在等某个事件（如 I/O 完成），不能运行

状态转换：运行 ↔ 就绪（由调度决定），运行 → 阻塞（发起 I/O），阻塞 → 就绪（I/O 完成）。

还有 僵尸态（zombie）：进程已退出但父进程还没调用 `wait()` 回收。

进程控制块

OS 用 PCB（Process Control Block）记录每个进程的信息。以 xv6 为例，`proc` 结构体包含：进程状态、页表指针、内核栈、打开的文件列表、当前目录等。`context` 结构体保存寄存器（用于上下文切换）。

进程 API（UNIX）

UNIX 用 `fork()`、`exec()`、`wait()`
三个系统调用来创建和管理进程。这套设计看起来奇怪，组合起来却非常强大。

fork()

`fork()` 创建调用进程的一个几乎完全相同的副本。调用一次，返回两次：

- 父进程收到子进程的 PID（正整数）
- 子进程收到 0

```
int rc = fork();
if (rc == 0) {
    //
} else {
    // rc  PID
}
```

子进程从 `fork()` 返回处继续执行，不是从头开始。父子谁先运行取决于调度器，不确定。

wait()

`wait()` 让父进程等子进程结束。如果父进程先调度到，`wait()`
会阻塞直到子进程退出；如果子进程先跑完，`wait()` 立即返回。这保证了确定性的执行顺序。

exec()

`exec()`
用一个全新的程序替换当前进程的代码和数据。调用成功后不会返回——因为当前程序已经被替换了。

```
//  exec
execvp("wc", args);  //  wc
//
```

为什么要分离 fork 和 exec

分成两步是关键设计。在 `fork()` 之后、`exec()` 之前，子进程可以做任意操作——最常见的就是 I/O 重定向：

```
int rc = fork();
if (rc == 0) {
    close(STDOUT_FILENO);              //
    open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  // fd 1
    execvp("wc", args);               // wc
}
```

`open()` 会分配最小的可用文件描述符。关掉 stdout（fd 1）后再 `open()`，新文件自然拿到 fd 1。`wc` 完全不知道输出被重定向了。

shell 的工作原理

shell 本质就是个循环：

1. 打印提示符
2. 读取用户输入的命令
3. `fork()` 出子进程
4. 子进程 `exec()` 执行命令
5. 父进程 `wait()` 等子进程结束
6. 回到步骤 1

pipe()

`pipe()` 系统调用把一个进程的输出连接到另一个进程的输入。shell 里的 `|` 就靠它实现：`grep foo file | wc -l` 把 `grep` 的 stdout 接到 `wc` 的 stdin。

其他 API

- `kill()`：给进程发信号（如 `SIGINT`、`SIGTSTP`）
- 信号的处理：进程可以用 `signal()` 自定义信号处理函数

受限直接执行

OS 需要解决两个问题：高效（不能什么都自己来）和
控制（不能让进程为所欲为）。受限直接执行（Limited Direct Execution，LDE）是解决方案。

用户模式与内核模式

硬件提供两种运行模式：

- 用户模式（user mode）：进程在这里运行，不能直接执行特权操作（如 I/O、修改页表）
- 内核模式（kernel mode）：OS 在这里运行，什么都能干

进程想做特权操作时必须通过系统调用。

系统调用与 `trap`

系统调用的本质是一条特殊指令——`trap`。执行 trap 时：

1. 硬件把寄存器保存到该进程的内核栈
2. 跳转到预设的 `trap` 处理程序
3. 切换到内核模式
4. OS 执行请求的操作
5. 执行 `return-from-trap`，恢复寄存器，回到用户模式继续执行

`trap` 表

OS 在启动时设置 `trap` 表（trap table），告诉硬件各种异常和系统调用对应的处理程序地址。这是在内核模式下做的——用户程序不能修改 trap 表，否则就能跳到任意内核代码了。

每个系统调用有一个系统调用号，用户程序设置好调用号后执行 trap，OS
查表找到对应处理函数执行。这样用户程序不能直接指定跳转地址，只能请求已注册的服务。

进程切换

OS 怎么从一个正在跑的进程手里拿回 CPU？靠 时钟中断（timer interrupt）。

时钟设备每隔几毫秒产生一次中断，当前进程被打断，OS 的中断处理程序开始执行。此时 OS
可以决定：继续跑当前进程，还是切换到别的进程。

上下文切换

如果 OS 决定切换进程，执行上下文切换（context switch）。注意这里有两次寄存器保存/恢复：

1. 硬件：trap 时自动把用户寄存器存到当前进程的内核栈
2. OS：把当前进程的内核寄存器存到它的 PCB，再从目标进程的 PCB 恢复内核寄存器

切换到目标进程的内核栈后，执行 return-from-trap 就回到了目标进程的用户空间。

以 xv6 的 `swtch()` 为例，它只保存/恢复被调用者保存的寄存器（callee-saved），因为调用者保存的寄存器已经由调用约定处理了。

LDE 协议总结

阶段	操作
启动	OS 初始化 trap 表，启动时钟中断
运行进程	OS 设好寄存器，return-from-trap 到用户模式
系统调用	trap → 内核处理 → return-from-trap
时钟中断	中断 → OS 决定是否上下文切换 → return-from-trap 到目标进程

调度：基础策略

这一章介绍的简单调度算法只是铺垫。真正实用的调度算法是下一章的 MLFQ。

评价指标

- 周转时间 = 完成时间 − 到达时间（越小越好）
- 响应时间 = 首次运行时间 − 到达时间（对交互式系统更重要）

这两个指标有内在冲突：优化周转时间的策略通常损害响应时间，反之亦然。

简单算法速览

算法	策略	问题
FIFO	先到先服务	护航效应：长作业卡住后面的短作业
SJF	最短作业优先	不可抢占，新来的短作业得等
STCF	最短完成时间优先	SJF 的抢占版，但需要预知作业长度（不现实）
RR	轮转，按时间片轮流跑	响应时间好，但周转时间差

这些算法要么需要知道作业长度（现实中不可能），要么只能在单一指标上表现好。下一章的 MLFQ 在不需要先验知识的情况下同时兼顾两个指标。

多级反馈队列（MLFQ）

MLFQ 是实际操作系统中最广泛使用的调度算法。它不需要预知作业长度，既能给短作业（交互式进程）低延迟，又能让长作业（CPU 密集型）最终完成。

基本结构

MLFQ 有多个队列，每个队列对应一个优先级。高优先级队列里的进程先运行。不同队列可以设置不同的时间片——高优先级队列用短时间片（如 10ms），低优先级队列用长时间片（如 100ms 甚至更长）。

五条规则

1. 优先级高的先运行
2. 优先级相同的轮转（Round-Robin）
3. 新进程放到最高优先级队列
4. 进程在某一层累计使用完该层的时间配额后，降到下一层（不管中间是否主动让出 CPU）
5. 每隔一段时间 S，把所有进程重新提到最高优先级

为什么这样设计

规则 3 实现了"乐观假设"——每个新来的进程先当短作业对待。如果它确实是短作业，很快就能跑完，响应时间很好；如果是长作业，它会逐步降级。

规则 4 是反作弊机制。如果只看"用完时间片才降级"，恶意进程可以在时间片快用完时主动让出 CPU（比如发一个无意义的 I/O），下次就能继续在高优先级运行。用累计时间代替单次时间片，堵死了这个漏洞。

规则 5 解决饥饿问题。没有周期性提升的话，大量交互式进程会一直占据高优先级队列，CPU 密集型进程永远得不到执行。每隔 S 时间做一次全体提升（priority boost），所有进程回到最高队列重新竞争。S 太大，长作业会饿死；S 太小，交互式进程得不到足够优先级。

实际系统中的 MLFQ

Solaris 的分时调度类：管理员可以配一张表，指定每层的时间片长度、降级目标、提升频率。默认 60 个优先级队列。

FreeBSD 4.3 用衰减使用（decay-usage）公式动态计算优先级，基于进程使用了多少 CPU。

Windows 和 macOS 也使用类似 MLFQ 的机制。Windows 会在等待 I/O 完成后临时提升进程优先级。

参数调优

MLFQ 需要调的参数：队列数量、每层的时间片、提升频率S。没有万能配置，不同工作负载需要不同参数。有些系统（如Solaris）允许管理员通过配置表调整，有些系统用数学公式自动计算。

多处理器调度

单 CPU 的调度已经不够，现代机器都是多核的。多处理器调度要面对新的硬件问题。

缓存与一致性

每个 CPU 都有自己的缓存（L1/L2，有些 L3 共享）。同一份数据可能在多个 CPU 的缓存中各有一份，某个 CPU 修改数据后，其他 CPU 缓存里的旧值就不对了。

硬件通过总线窥探（bus snooping）解决这个问题：每个缓存监控总线上的内存操作，发现有人写了自己缓存中存在的数据时，立即将自己的副本标记为无效或更新。这对上层软件基本透明。

缓存亲和性

一个进程在某个 CPU 上跑过之后，该 CPU 的缓存里已经有它的数据了。如果下次调度到同一个 CPU，缓存命中率高，性能好。调度器应尽量让进程保持在同一个 CPU 上，这叫缓存亲和性（cache affinity）。

两种调度方案

SQMS（单队列多处理器调度）

所有进程放在一个全局队列里，每个 CPU 从里面取。

- 优点：简单，负载自动均衡
- 缺点：全局队列需要加锁，CPU 越多竞争越激烈，扩展性差。缓存亲和性也难保证（进程可能被随机分到不同 CPU）

MQMS（多队列多处理器调度）

每个 CPU 有自己的队列，进程创建时分配到某个队列。

- 优点：无需全局锁，扩展性好。天然有缓存亲和性（进程固定在一个 CPU）
- 缺点：可能负载不均。如果某个 CPU 的进程都结束了，它就空转了

解决负载不均的办法是工作窃取（work stealing）：空闲 CPU 偶尔看一眼别的队列，如果人家进程多就偷几个过来。检查频率需要权衡——太频繁增加开销，太少则负载失衡时间长。

Linux 调度器

Linux 内核里同时存在多种调度器：

- CFS（Completely Fair Scheduler）：当前默认，用红黑树按虚拟运行时间排序
- BFS（Brain Fuck Scheduler）：单队列设计，用最早合适截止时间（EEVDF）思路
- O(1) 调度器：老版本，多队列，基于优先级

不同调度器对不同工作负载各有优劣。

地址空间

从物理内存到虚拟化

早期系统没有内存抽象，程序直接访问物理内存。后来为了跑多个程序，需要隔离——一个进程不能读写另一个进程的内存，也不能动 OS 的内存。

OS 给每个进程提供一个地址空间（address space）的抽象：进程以为自己独占一大块连续内存，实际上 OS 和硬件在背后做了地址翻译。

地址空间的布局

一个典型的地址空间从低到高：

- 代码段（code）：程序指令，固定大小，只读
- 堆（heap）：动态分配的内存（`malloc`），从低地址向高地址增长
- 栈（stack）：函数调用的局部变量和返回地址，从高地址向低地址增长

堆和栈相向增长，中间是未使用的空间。

虚拟内存的三个目标

- 透明：进程完全不知道内存被虚拟化了，以为自己有独立的一大块内存
- 高效：时间上不能太慢（靠硬件支持如 TLB），空间上不能浪费太多
- 保护/隔离：进程之间互不干扰，进程也不能破坏 OS

内存 API

栈内存 vs 堆内存

- 栈内存：编译器自动管理，函数里声明的局部变量在函数返回时自动释放
- 堆内存：程序员手动管理，`malloc()` 分配，`free()` 释放

核心 API

```
void *malloc(size_t size);    //  size  void
void free(void *ptr);          //  malloc
void *calloc(size_t n, size_t size);   //
void *realloc(void *ptr, size_t size); //
```

`malloc()` 的参数是字节数，通常配合 `sizeof` 使用：`malloc(sizeof(int) * 10)`。`sizeof` 在编译期求值（变长数组除外）。

`free()` 只需要指针，不需要传大小——分配器内部记录了每块的大小。

常见错误

错误	说明
忘记分配	直接用未分配的指针，如 `strcpy(dst, src)` 但 `dst` 没 malloc
分配不够	少分了一个字节（比如忘了字符串末尾的 `\0`），缓冲区溢出
忘记初始化	`malloc` 不清零，读到随机值
忘记释放	内存泄漏。短命程序无所谓，长期运行的程序（如服务器、OS）会慢慢耗光内存
用完还在用	悬空指针（dangling pointer），`free` 之后又去读写
重复释放	double free，结果未定义，通常导致崩溃
释放错误地址	传给 `free` 的不是 `malloc` 返回的地址

底层机制

`malloc`/`free` 是库函数，不是系统调用。它们在用户空间管理内存。

底下的系统调用是 `brk`/`sbrk`（调整堆分界点）和 `mmap`（映射匿名页或文件）。一般不应直接调用 `brk`/`sbrk`。

所以内存管理分两层：OS 给进程分页级别的内存，`malloc` 库在这些页里面做细粒度的分配和回收。

分页

分页是现代 OS 管理内存的核心机制。把物理内存切成固定大小的块（页帧，通常 4KB），进程的地址空间也按同样大小切成页（page）。任意一页可以放在任意一个页帧里。

页表

OS 为每个进程维护一个页表（page table），记录虚拟页到物理页帧的映射。

虚拟地址 = VPN（虚拟页号）+ offset（页内偏移）

翻译过程：从虚拟地址取出 VPN → 查页表得到 PFN → 拼上 offset → 物理地址。

以 64 字节地址空间、16 字节页为例：虚拟地址 21 = 二进制 `010101`，高 2 位 `01` 是 VPN=1，低 4 位 `0101` 是偏移=5。查页表 VPN 1 → PFN 7，物理地址 = 7×16+5 = 117。

页表项（PTE）

页表的每一项包含多个标志位：

标志位	含义
有效位（valid）	该页是否被进程使用。未使用的虚拟页标记无效，访问会触发 trap
保护位（protection）	读/写/执行权限
存在位（present）	该页是否在物理内存中（还是被换出到磁盘了）
脏位（dirty）	页被写过，换出时需要回写磁盘
参考位（reference/accessed）	页被访问过，用于页替换算法

页表太大的问题

32 位地址空间 + 4KB 页 = <font color="#D9534F">2<super>20</super></font> 个页表项。每项 4 字节的话，单个页表就 4MB。系统里跑 100 个进程就是 400MB
纯页表开销。加上每次内存访问都要多一次查页表的访问，性能也受影响。

这两个问题分别由 TLB（下一章）和多级页表来解决。

TLB

页表在内存里，每次虚拟地址翻译都要先查内存里的页表，再访问实际数据——等于每次访存变成两次。TLB（Translation Lookaside Buffer）是 CPU 里的硬件缓存，专门缓存最近用过的页表项。

工作流程

1. CPU 拿到虚拟地址，提取 VPN
2. 查 TLB
3. 命中：直接拿到 PFN，拼上偏移，访问物理内存（快）
4. 未命中：去内存查页表，把结果装进 TLB，重试指令

TLB 命中率极高（典型 > 99%），靠的是时间局部性（刚访问的数据很可能再次访问）和空间局部性（相邻数据很可能被访问）。

硬件管理 vs 软件管理

硬件管理 TLB（x86 等 CISC）：TLB 未命中时硬件自动查页表、填 TLB，OS 不参与。硬件需要知道页表的确切格式和内存位置（通过 CR3 寄存器）。

软件管理 TLB（MIPS 等 RISC）：TLB 未命中触发异常，由 OS 的 trap handler 来查页表、用特殊指令填 TLB。更灵活（OS 可以用任意页表格式），但稍慢。

TLB 内容

每个 TLB 条目包含：VPN、PFN、有效位、保护位、脏位等。

TLB 通常是全相联（fully associative）的——任何条目可以在 TLB 的任何位置。查找时并行比较所有条目。

上下文切换时怎么办

不同进程的虚拟页号映射到不同物理页，TLB 里的旧映射对新进程无意义。

两种方案：

- 清空 TLB（flush）：切换时把 TLB 全部清掉。简单但代价大，新进程刚开始全是未命中
- ASID（Address Space Identifier）：TLB 条目带一个 ASID 字段（类似进程标识，通常 8 位），不同进程的条目可以共存。比清空高效得多

替换策略

TLB 满了要踢掉一个条目：

- LRU：踢最久没用过的。工作集稳定时效果好

- 随机：随便踢一个。避免 LRU 的最差情况（比如循环访问模式），实现简单

更小的页表

线性页表太大（32 位地址空间 + 4KB 页 → 每进程 4MB），需要压缩。

方案一：更大的页

页大小从 4KB 变 16KB，页表项数量少 4
倍。但会导致内部碎片——每页内浪费的空间变多。这不是好方案。

方案二：分页与分段结合

给代码段、堆、栈各维护独立的页表，用段基址寄存器指向各自的页表。未使用的地址空间区域不需要页表——节省了地址空间中间那大片空洞的页表项。

问题：分段不够灵活，外部碎片又回来了。实际系统很少用。

方案三：多级页表

这是现代系统的标准方案。核心思路：把页表本身也分页。

先加一层页目录（page directory）。页目录的每一项指向一页页表。如果某一项覆盖的虚拟地址范围内没有任何有效页，那这个页目录项标记为无效，对应的那一页页表根本不需要分配。

举例

16KB 地址空间，64B 的页 → 256 个页表项。假设每页能放 16 个页表项 → 页表需要 16 页。页目录有 16 项，每项管一页页表。

如果只用了地址空间两头（代码+堆在低端，栈在高端），中间的页目录项是无效的，对应的页表不分配。

多级

32 位或 64 位地址空间一层不够，可以套多层——页目录指向页目录，直到最后一层指向页表页。

权衡

好处：

- 只为实际使用的虚拟地址范围分配页表空间
- 页表页本身是页大小，方便管理

代价：

- TLB 未命中时需要多次内存访问（两级就要两次查找，三级三次）
- 实现复杂度更高

方案四：反向页表

不按虚拟页号索引，而是按物理页帧号索引。一个系统只需要一个反向页表，每个物理页帧一个条目，记录哪个进程的哪个虚拟页映射到这里。

查找需要遍历或哈希。PowerPC 用过这种设计。

超越物理内存：机制

物理内存不够用时，OS 把暂时不用的页放到磁盘上（交换空间），用到时再换回来。让进程以为自己有比实际物理内存大得多的地址空间。

交换空间

磁盘上专门划一块区域存放被换出的页，叫交换空间（swap space）。OS 以页为单位在交换空间和物理内存之间来回搬数据。

注意：程序代码页不需要写回交换空间，因为可以直接从程序的二进制文件重新加载。

存在位

页表项里加一个存在位（present bit）：

- 1 = 页在物理内存中，正常翻译
- 0 = 页不在内存里（被换出到磁盘或者还没加载进来）

访问一个存在位为 0 的页 → 触发页错误（page fault）。

页错误处理

页错误由 OS 的页错误处理程序处理（不管是硬件还是软件管理的 TLB，页错误都交给 OS）：

1. 从 PTE 里读出该页在磁盘上的地址
2. 找一个空闲物理页帧（如果没有，先用页替换算法踢掉一页）
3. 发起磁盘 I/O，把页读进物理内存
4. 更新 PTE：设置 PFN，存在位改为 1
5. 重试触发页错误的那条指令

磁盘 I/O 期间，触发页错误的进程被阻塞，OS 调度其他进程运行。这就是多道程序充分利用 CPU 的方式。

硬件控制流

```
VPN =
 TLB:
   →  →
   → :
    PTE  → SEGFAULT
    PTE present=0→ PAGE_FAULTOS
    PTE  →  TLB →
```

水位线机制

OS 不会等内存完全用光才开始换出。设置两个阈值：

- 低水位线（LW）：空闲页低于此值时，后台的交换守护进程（swap daemon / page daemon）开始换出页面
- 高水位线（HW）：换出到空闲页达到此值时停止

批量换出（聚集写入）能利用磁盘顺序写的高吞吐，比每次换一页高效得多。

页替换策略

物理内存不够时需要换出页面，选谁换出是页替换策略的工作。选错了代价极大——磁盘访问比内存慢 10000 到 100000 倍。

评价指标

$$AMAT = P_{Hit} \times T_M + P_{Miss} \times T_D$$

平均内存访问时间（AMAT）= 命中率 × 内存延迟 + 未命中率 × 磁盘延迟。因为磁盘太慢，即使很小的未命中率也会严重拉高 AMAT。

几种策略

最优策略（OPT/MIN）：踢掉未来最久不会被访问的页。理论最优但不可实现——需要预知未来，只用来做比较基准。

FIFO：踢掉最早进入内存的页。简单但效果差。存在 Belady 异常——增加物理内存反而可能增加缺页次数。

随机：随便踢一个。有时比 FIFO 好，但不稳定。

LRU：踢掉最近最久未使用的页。利用历史信息近似预测未来。在大多数工作负载下接近最优。不会有 Belady 异常（LRU 属于栈算法）。

LRU 的近似实现——时钟算法

精确维护 LRU 的访问时间代价太高。实际系统用时钟算法（Clock Algorithm）近似：

- 所有页围成一个环形链表，维护一个时钟指针
- 每页有一个使用位（use bit / reference bit），页被访问时硬件自动置 1
- 需要替换时，时钟指针扫描：
  - 使用位 = 1 → 清零，跳过
  - 使用位 = 0 → 选中它换出

效果：最近被用过的页使用位为 1，会被跳过（得到"第二次机会"）。长时间没用的页使用位早被清零，会被选中。

优化：优先换出干净页（脏位 = 0），因为不用写回磁盘。

工作负载与策略选择

- 无局部性（完全随机访问）：所有策略表现相同
- 80-20 负载（80% 访问集中在 20% 页面）：LRU 明显优于 FIFO 和随机

- 循环扫描（依次访问 N 个页，反复循环）：如果 N 大于物理页数，LRU 和 FIFO 都 100% 未命中（最差情况），随机反而更好

抖动

如果进程的活跃页面总量超过了物理内存容量，系统不断换入换出，大量时间花在磁盘 I/O 上,叫抖动（thrashing）。

解决方案：

- 准入控制：不允许运行更多进程，让一部分进程的工作集完全放入内存
- `OOM Killer`：直接杀掉占内存最多的进程，释放内存（Linux 的做法）

第二部分：并发

并发与线程

线程

线程是比进程更轻量的执行单元。同一进程内的线程共享地址空间（代码、堆、全局变量），但各有自己的寄存器和栈。

每个线程由线程控制块（TCB）描述，线程切换时保存/恢复的是 TCB
里的寄存器状态。因为共享地址空间，线程切换不用切换页表，比进程切换轻得多。

多线程进程的地址空间里有多个栈，每个线程一个，分布在地址空间的不同区域。

为什么需要线程

- 并行：多核 CPU 上同时执行，提高吞吐
- 避免 I/O 阻塞：一个线程阻塞等
  I/O，其他线程继续跑。用多进程也能做到，但线程共享地址空间，通信更方便

共享数据的问题

多线程操作共享变量时会出问题。经典例子：两个线程同时 `counter++`。

`counter++` 编译后是三条指令：

1. 把 `counter` 从内存读到寄存器
2. 寄存器加 1
3. 把结果写回内存

如果两个线程交错执行这三条指令，最终结果可能只加了 1 而不是 2。

核心概念

- 竞态条件（race condition）：结果取决于线程执行的时序
- 临界区（critical section）：访问共享资源的代码段
- 互斥（mutual exclusion）：保证同一时刻只有一个线程进入临界区
- 不确定性（indeterminate）：同样的输入可能产生不同的输出——并发 bug 的根源
- 原子性（atomicity）：一组操作要么全部完成要么全都没做，中间不会被打断

线程 API（POSIX）

创建线程

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
```

- `thread`：输出参数，线程标识符
- `attr`：线程属性，一般传 `NULL`（默认属性）
- `start_routine`：线程执行的函数
- `arg`：传给函数的参数（`void *` 类型，需要自己转型）

等待线程

```
int pthread_join(pthread_t thread, void **retval);
```

阻塞等目标线程结束。`retval` 接收线程的返回值。不是所有线程都需要join——后台干活的线程可以不管。

注意：不要返回指向线程栈上变量的指针——线程结束后栈被回收,指针悬空。

互斥锁

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
//  pthread_mutex_init(&amp;lock, NULL);

pthread_mutex_lock(&amp;lock);
//
pthread_mutex_unlock(&amp;lock);
```

`lock` 和 `unlock` 必须配对。加锁失败会阻塞等待。每次调用都必须检查返回值——虽然很多人不检查。

条件变量

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

pthread_cond_wait(&amp;cond, &amp;lock);   // +
pthread_cond_signal(&amp;cond);         //
```

`wait` 必须在持有锁的状态下调用。等待条件时必须用 `while` 循环（不是if），因为被唤醒后条件可能已经被其他线程改变了。

编译

编译时加 `-pthread` 链接线程库。

锁

锁提供互斥——同一时刻只有一个线程能持有锁。拿不到锁的线程要么自旋等待，要么休眠。

评价一个锁实现看三点：正确性（是否真的互斥）、公平性（是否有线程饿死）、性能（开销多大）。

关中断

最简单的方案：进临界区前关中断，出来再开。

问题一堆：只对单处理器有用；恶意程序可以关中断后死循环独占 CPU；关中断期间 I/O 信号全丢；现代多处理器上根本不可行。OS 内部访问自己的数据结构时偶尔会用。

自旋锁

Test-And-Set（原子交换）

```
int TestAndSet(int *old_ptr, int new) {
    int old = *old_ptr;
    *old_ptr = new;
    return old;  //
}
```

锁的实现：`while (TestAndSet(&flag, 1) == 1);`（自旋等待 flag 变 0）。解锁时 `flag = 0`。

正确，但不公平（饿死可能），单 CPU 上性能差（自旋浪费整个时间片）。

Compare-And-Swap（CAS）

```
int CompareAndSwap(int *ptr, int expected, int new) {
    int actual = *ptr;
    if (actual == expected) *ptr = new;
    return actual;
}
```

只有 `*ptr == expected` 时才更新。做锁效果和 TAS 相同，但 CAS 更通用——是无锁数据结构的基础。

Fetch-And-Add → 票锁

```
int FetchAndAdd(int *ptr) {
    int old = *ptr;
    *ptr = old + 1;
    return old;
}
```

票锁（ticket lock）：每个线程取号（`FetchAndAdd(&ticket)`），然后等轮到自己的号（`while (turn != myticket)`）。保证公平——先来先服务。

自旋的问题

自旋锁在等待时疯狂烧 CPU。单 CPU
上尤其浪费：持有锁的线程被抢占后，其他线程自旋一整个时间片。

简单改进：拿不到锁就 `yield()`（主动让出 CPU）。比纯自旋好，但线程多时仍有大量无效切换。

基于队列的休眠锁

真正高效的方案：拿不到锁就休眠，锁释放时被唤醒。

Solaris 提供 `park()`（休眠）和 `unpark(tid)`（唤醒指定线程）。实现要点：

1. 用一个 guard 自旋锁保护锁内部的队列操作（只自旋极短时间）
2. 拿不到锁时把自己加入等待队列，然后 `park()` 休眠
3. 释放锁时如果队列非空，`unpark()` 唤醒队首线程
4. 有个"唤醒/等待竞争"问题：线程 A 准备 park 但还没睡，线程 B 已经 unpark 了 → A 永远睡过去。Solaris 用 `setpark()` 解决——调用后如果被 unpark，下次 `park()` 立即返回

Linux futex

Linux 的 `futex`（fast userspace mutex）在用户空间和内核之间分工：

- `futex_wait(addr, expected)`：如果 `*addr == expected` 就休眠，否则立即返回
- `futex_wake(addr)`：唤醒一个在 addr 上等待的线程

glibc 的 nptl 锁实现用一个 32 位整数：最高位表示锁是否被持有，其余位是等待者计数。无竞争情况下一次原子操作就够了（快路径），有竞争才走系统调用休眠（慢路径）。

两阶段锁

先自旋一小会儿（赌锁很快释放），如果还拿不到再休眠。结合了自旋锁的低延迟和休眠锁的低浪费。Linux 的 futex 锁就是这个思路。

基于锁的并发数据结构

给数据结构加锁的方式很影响性能。粗粒度锁（一把大锁保护整个结构）简单但扩展性差；细粒度锁复杂但并发度高。

并发计数器

简单方案

一把锁保护一个全局计数器。正确但完全串行，线程越多越慢。

懒惰计数器（sloppy counter）

每个 CPU 一个本地计数器（带各自的锁），加上一个全局计数器（带全局锁）。

- 线程在自己 CPU 的本地计数器上操作——不同 CPU 之间无竞争
- 本地计数器累积到阈值 S 后，一次性转移到全局计数器，本地清零

S 越小，全局值越精确但转移越频繁（像精确计数器一样慢）。S 越大，转移越少但全局值越不精确。S = 1024 时，4 个 CPU 的性能已经接近无锁单线程。

并发链表

最直接的做法是整个链表一把锁。性能一般但够用。

Hand-over-hand locking（每个节点一把锁，遍历时逐步交接）理论上并发度高，但加锁/解锁的开销太大，实际很少比一把大锁快。

并发队列（Michael-Scott 队列）

头部一把锁，尾部一把锁，入队和出队互不阻塞。

关键技巧：用一个哨兵节点（dummy node）分开头尾操作。入队操作只碰尾部，出队操作只碰头部。没有哨兵的话，队列只有一个元素时头尾指向同一个节点，两把锁就冲突了。

并发哈希表

最简单也最实用的方案：每个桶一把锁（per-bucket lock）。不同桶的操作完全并行。

扩展性好。线程数增加时吞吐近乎线性增长——因为大部分操作落在不同的桶里。这是实际系统中最常见的并发哈希表设计。

条件变量

锁解决互斥，条件变量解决等待——线程需要等某个条件成立才能继续。

基本操作

```
pthread_cond_wait(cond, mutex);   //  mutex  mutex
pthread_cond_signal(cond);        //  cond
pthread_cond_broadcast(cond);     //
```

等待时必须用 while，不能用 if

```
pthread_mutex_lock(&amp;m);
while (!condition) {              //  while
    pthread_cond_wait(&amp;cond, &amp;m);
}
//
pthread_mutex_unlock(&amp;m);
```

原因：Mesa 语义。`signal` 只是一个提示（"条件可能满足了"），被唤醒的线程不一定能立即运行，等它拿到锁开始跑的时候，条件可能又被别的线程改掉了。所以唤醒后必须重新检查条件。

（Hoare 语义保证唤醒后立即运行，条件一定成立。但几乎没有系统实现它。）

生产者/消费者（有界缓冲区）

经典并发问题。生产者往缓冲区放东西，消费者从里面取。缓冲区满了生产者等，空了消费者等。

错误写法演进

单条件变量 + if：消费者 A 被生产者唤醒，但消费者 B 在 A 之前先跑了并取走了数据，A 拿到锁时缓冲区又空了——坏了。

单条件变量 + while：条件检查没问题了，但消费者可能唤醒消费者（不是生产者），大家全在睡觉——死了。

最终方案

两个条件变量：`empty` 和 `fill`。

```
//
int buffer[MAX];
int count = 0, fill_ptr = 0, use_ptr = 0;

//
pthread_mutex_lock(&amp;mutex);
while (count == MAX)
    pthread_cond_wait(&amp;empty, &amp;mutex);  //
put(value);
pthread_cond_signal(&amp;fill);             //
pthread_mutex_unlock(&amp;mutex);

//
pthread_mutex_lock(&amp;mutex);
while (count == 0)
    pthread_cond_wait(&amp;fill, &amp;mutex);   //
int val = get();
pthread_cond_signal(&amp;empty);            //
pthread_mutex_unlock(&amp;mutex);
```

两个条件变量保证：消费者只唤醒生产者，生产者只唤醒消费者。

覆盖条件

有时候不知道该唤醒哪个等待者。比如内存分配器：线程 A 等 100 字节，线程 B 等 10 字节，释放了 50 字节应该唤醒谁？

保守方案：用 `pthread_cond_broadcast` 唤醒所有人，每个人自己检查条件。被称为覆盖条件（covering condition）。代价是可能唤醒很多不必要的线程，但保证正确。

信号量

信号量（semaphore）是 Dijkstra 提出的同步原语。一个信号量就是一个带等待队列的整数。

基本操作

```
sem_init(&amp;s, 0, value);  //  value 0
sem_wait(&amp;s);             //  1 &lt; 0
sem_post(&amp;s);             //  1
```

信号量的值为负数时，绝对值就是等待队列里的线程数。

当锁用

初始值设为 1（二值信号量）。`sem_wait` 拿锁，`sem_post` 放锁。

```
sem_t lock;
sem_init(&amp;lock, 0, 1);
sem_wait(&amp;lock);    //  1→0
//
sem_post(&amp;lock);    //  0→1
```

当条件变量用

初始值设为 0。`sem_wait` 等待，`sem_post` 通知。

用来做线程间排序：父线程 `sem_wait(&s)` 等子线程完成，子线程完成后 `sem_post(&s)`。

生产者/消费者

需要三个信号量：

```
sem_t empty, full, mutex;
sem_init(&amp;empty, 0, MAX);  //
sem_init(&amp;full, 0, 0);     //
sem_init(&amp;mutex, 0, 1);    //
```

```
//
sem_wait(&amp;empty);     //  mutex
sem_wait(&amp;mutex);
put(value);
sem_post(&amp;mutex);
sem_post(&amp;full);      //

//
sem_wait(&amp;full);      //  mutex
sem_wait(&amp;mutex);
int val = get();
sem_post(&amp;mutex);
sem_post(&amp;empty);     //
```

`mutex` 必须在 `empty`/`full` 的里面。如果反过来，消费者持有 mutex 等 full，生产者持有 empty 等 mutex → 死锁。

读者-写者锁

多个读者可以同时读，写者需要独占。

```
typedef struct {
    sem_t lock;       //  readers
    sem_t writelock;  //
    int readers;
} rwlock_t;
```

第一个读者进入时获取 `writelock`（挡住写者），最后一个读者离开时释放 `writelock`。

问题：持续有读者进入时写者会饿死。

哲学家就餐

5 个哲学家围坐，每人左右各一把叉子（共 5 把），吃饭需要两把。每把叉子是一个信号量。

如果都先拿左边再拿右边 → 环形等待 → 死锁。

解法：让最后一个哲学家先拿右边再拿左边（打破环形等待）。

用锁和条件变量实现信号量

```
typedef struct {
    int value;
    pthread_cond_t cond;
    pthread_mutex_t lock;
} Zem_t;

void Zem_wait(Zem_t *s) {
    pthread_mutex_lock(&amp;s-&gt;lock);
    while (s-&gt;value &lt;= 0)
        pthread_cond_wait(&amp;s-&gt;cond, &amp;s-&gt;lock);
    s-&gt;value--;
    pthread_mutex_unlock(&amp;s-&gt;lock);
}

void Zem_post(Zem_t *s) {
    pthread_mutex_lock(&amp;s-&gt;lock);
    s-&gt;value++;
    pthread_cond_signal(&amp;s-&gt;cond);
    pthread_mutex_unlock(&amp;s-&gt;lock);
}
```

这个实现的 value 不会小于 0（和 Linux 的 `sem_wait` 语义略有不同，Dijkstra 原始定义允许负值）。

常见并发 Bug

并发 bug 分两大类：非死锁 bug 和死锁 bug。实际系统中非死锁 bug 占了大多数（约 97%）。

非死锁 Bug

原子性违反

"我以为这段代码会原子执行，但没有。"

MySQL 的例子：

```
//  1
if (thd-&gt;proc_info) {
    fputs(thd-&gt;proc_info, ...);
}

//  2
thd-&gt;proc_info = NULL;
```

线程 1 检查非空后、使用前，线程 2 可能把它置为 NULL → 空指针崩溃。

修复：用锁把检查和使用包成一个原子操作。

顺序违反

"我以为线程 A 一定在线程 B 之前执行某个操作。"

比如 `mThread` 在线程 1 里创建，线程 2 直接用它——但线程 2 可能先跑到。

修复：用条件变量强制顺序。

死锁

两个或更多线程互相等待对方持有的资源，谁也走不了。

4 个必要条件

死锁发生当且仅当以下四个条件同时成立：

1. 互斥：资源不能共享
2. 持有并等待：线程持有一个资源的同时等待另一个
3. 非抢占：资源不能被强制收回
4. 循环等待：A 等 B，B 等 A（或更长的环）

打破任何一个条件就能避免死锁。

预防策略

打破循环等待——锁排序

给所有锁定一个全局顺序，所有线程按相同顺序获取锁。比如总是先拿 L1 再拿 L2。

如果不知道两把锁谁先谁后（比如参数传入），可以用地址排序：地址小的先拿。

这是最实用的策略。Linux 内核的锁文档里就规定了锁的获取顺序。

打破持有并等待

在获取任何锁之前，先拿一把全局的 prevention 锁。这样要么一次拿到所有需要的锁，要么一个都没拿。

缺点：需要预先知道要用哪些锁，而且把所有获取操作串行化了。

打破非抢占——`trylock`

用 `pthread_mutex_trylock()`：拿到了返回 0，拿不到立即返回错误（不阻塞）。

```
top:
    lock(L1);
    if (trylock(L2) != 0) {
        unlock(L1);
        goto top;    //
    }
```

问题：两个线程可能反复同时获取/释放 → 活锁（livelock）。加随机延迟能缓解。

打破互斥——无锁编程

用 CAS 等原子操作代替锁：

```
void AtomicIncrement(int *val, int amount) {
    do {
        int old = *val;
    } while (CompareAndSwap(val, old, old + amount) != old);
}
```

更复杂的例子：无锁链表插入用 CAS 保证只有一个线程成功插入头部，失败的线程重试。

无锁编程避免了死锁，但编写困难，而且会遇到 ABA 问题（值被改成别的又改回来，CAS 检测不到）和活锁。

其他方法

死锁避免（Dijkstra 银行家算法）：调度器根据线程的资源需求规划执行顺序，避开死锁状态。实际系统基本不用——需要预知所有资源需求，并发度也严重受限。

检测与恢复：允许死锁发生，定期检测资源图中的环，发现后杀进程或回滚。数据库系统常用这个方法。

基于事件的并发

除了线程，还有另一种并发模型：事件循环（event loop）。Node.js、Redis 都用这种方式。

事件循环

```
while True:
    events = getEvents()    #
    for e in events:
        processEvent(e)     #
```

核心思路：单线程，一次处理一个事件，不需要锁（单 CPU 下没有并发访问共享数据的问题）。

select()/poll()

事件循环需要知道哪些 I/O 就绪了。POSIX 的 `select()` / `poll()` 提供这个能力：

```
int select(int nfds, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
```

传入一组文件描述符，`select` 阻塞直到其中某些就绪（可读/可写/异常），返回就绪的数量。

应用只在事件就绪后才调用 `read()`/`write()`，不会阻塞。

阻塞操作的问题

事件循环最怕阻塞调用。一旦某个事件处理程序调用了阻塞的系统调用（比如读磁盘），整个服务器都停下了。

解决方案：异步 I/O（AIO）。

```
struct aiocb aio;
aio.aio_fildes = fd;
aio.aio_buf = buf;
aio.aio_nbytes = size;
aio.aio_offset = offset;
aio_read(&amp;aio);          //

//
int status = aio_error(&amp;aio);  //  EINPROGRESS
```

也可以用信号通知完成，但信号处理本身又引入并发问题。

状态管理

线程模型里，每个线程的局部状态自然存在栈上。事件模型里，异步操作返回时需要知道之前执行到哪了——必须手动管理状态。

这叫续体（continuation）：把回调需要的上下文打包保存，异步操作完成后用它继续执行。代码比线程版复杂不少。

多核上的局限

单线程事件循环在多核机器上无法利用所有 CPU。要想用多核，就得跑多个事件循环（每核一个），一旦它们之间有共享数据，又需要加锁——事件模型的简洁性就打了折扣。

还有一个隐性阻塞问题：缺页。如果事件处理程序访问的内存页不在物理内存中，触发页错误，整个线程被阻塞等磁盘 I/O，事件循环无法推进。这一点是程序员无法完全控制的。

第三部分：持久化

I/O 设备

系统架构

CPU 和内存通过内存总线连接（最快）。显卡等高性能设备接在 I/O 总线（如 PCIe）上。磁盘、鼠标、键盘等慢设备接在外围总线（如 USB、SATA）上。

越快的总线越短、越贵、设备越少。层次结构平衡了速度和成本。

设备接口

一个标准设备对外暴露三个寄存器：

- 状态寄存器：读取设备当前状态
- 命令寄存器：告诉设备执行什么操作
- 数据寄存器：传输数据

OS 通过读写这些寄存器与设备交互。

与设备交互的方式

显式 I/O 指令（x86 的 `in`/`out`）：特权指令，直接指定设备端口号读写。

内存映射 I/O（memory-mapped I/O）：设备寄存器被映射到物理地址空间的某个区域，OS 直接用 load/store 指令访问。

轮询 vs 中断

轮询（polling）：CPU 反复读状态寄存器，等设备就绪。简单粗暴，浪费 CPU。

中断（interrupt）：CPU 发出请求后去干别的事，设备完成后触发中断，OS 的中断处理程序被唤起。CPU 利用率高。

但中断不是万能的：

- 设备很快时，轮询可能比中断好——省了上下文切换的开销
- 网络流量大时中断太频繁（活锁——CPU 全在处理中断，没空干活）→ 混合方案：先轮询一会儿，没结果再转中断
- 中断合并（coalescing）：设备攒一批完成通知再触发一次中断，减少中断频率

DMA

没有 DMA 时，CPU 要一个字节一个字节地把数据搬到设备（PIO）。有了 DMA（Direct Memory Access），CPU 只需告诉 DMA 引擎：从内存地址 X 搬 N 字节到设备，然后 CPU 去忙别的。DMA 引擎独立完成数据传输，完事后中断通知 CPU。

设备驱动

OS 不可能为每种设备写专门代码。通用做法：定义一套抽象接口（如块设备接口：read block / write block），设备驱动程序负责把抽象接口翻译成具体设备的操作。

Linux 内核中设备驱动代码占 70% 以上，也是 bug 的重灾区。

文件系统不关心底下是 SCSI 盘还是 SATA 盘——它只对通用块层说"读第 X 块"，块层再交给对应的驱动处理。

磁盘驱动器

物理结构

磁盘由一个或多个盘片（platter）组成，每面有磁性涂层。盘片绕主轴高速旋转（7200~15000 RPM）。磁头在磁臂末端，悬浮在盘面上方读写数据。

每个盘面被划分为同心圆的磁道（track），每条磁道分为若干扇区（sector，通常 512B）。

I/O 时间

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

- 寻道时间（<font color="#D9534F">T_{seek}</font>）：磁头移动到目标磁道。包含加速、滑行、减速、稳定，典型 0.5~2ms（相邻磁道）到数毫秒（全程）
- 旋转延迟（<font color="#D9534F">T_{rotation}</font>）：等目标扇区转到磁头下方。平均等半圈。7200 RPM 的盘一圈约 8.3ms，平均旋转延迟约 4.15ms
- 传输时间（<font color="#D9534F">T_{transfer}</font>）：读写数据，取决于数据量和盘面线速度，通常最短

顺序 vs 随机

这是面试高频考点。以 Cheetah 15K 磁盘为例：

- 顺序读：~125 MB/s
- 随机读：~0.66 MB/s

差距约 200 倍。原因：随机读每次都要寻道+旋转，顺序读几乎不用。

磁盘调度

OS（或磁盘控制器）对 I/O 请求重排序以减少磁头移动：

SSTF（最短寻道时间优先）：选离当前磁头最近的请求。贪心，可能饿死远端请求。

SCAN / C-SCAN（电梯算法）：磁头从一端扫到另一端，沿途处理请求。C-SCAN 到达一端后直接跳回起点重新扫，对所有位置的请求更公平。

SPTF（最短定位时间优先）：同时考虑寻道和旋转延迟，选总定位时间最短的请求。效果最好，但 OS 通常不知道磁盘内部的精确布局，所以 SPTF 一般在磁盘固件里实现。

其他优化

- I/O 合并：相邻扇区的多个请求合并成一个大请求
- 磁道偏斜（track skew）：相邻磁道的 0 号扇区错开几个位置，避免跨磁道读时因旋转错过

- 磁盘缓存（track buffer）：读整条磁道到缓存。写策略分两种：write back（写入缓存就返回，快但断电丢数据）和 write through（写到盘面才返回，安全但慢）

文件与目录

文件

文件就是一个字节数组，由inode 号唯一标识。OS
不关心文件内容的格式——它只存字节，怎么解读是应用的事。

目录

目录也是一种文件，里面存的是 (名字, inode 号)
的列表。通过目录的层级嵌套构成整棵文件系统树，根目录是 `/`。

文件操作 API

打开与关闭

```
int fd = open("foo.txt", O_RDONLY);  //
// ...  fd  ...
close(fd);
```

`open()` 返回的文件描述符（fd）是一个整数。每个进程有自己的 fd 表。fd 0/1/2 分别是 stdin/stdout/stderr。`open()` 总是分配当前最小的可用 fd。

读写与偏移

```
ssize_t read(int fd, void *buf, size_t count);   //
ssize_t write(int fd, const void *buf, size_t count);
off_t lseek(int fd, off_t offset, int whence);   //
```

每个 fd 有一个当前偏移量（offset），`read()`/`write()` 后自动前移。`lseek()` 可以随意移动偏移量。注意 `lseek` 只改内存中的偏移变量，不会触发磁盘寻道。

fsync

```
fsync(fd);    //
```

普通 `write()` 可能只到了内核缓冲区，断电会丢。数据库等需要持久性保证的场景必须调 `fsync()`。要把目录项也持久化，还得对目录 fd 也调一次 `fsync()`。

rename

```
rename("tmp_file", "target_file");   //
```

rename 是原子的。安全更新文件的惯用模式：先写入临时文件 → `fsync` 临时文件 → `rename` 覆盖目标文件。这样保证要么看到完整的新文件，要么看到完整的旧文件。

stat

```
struct stat st;
stat("foo.txt", &amp;st);   //
```

返回 inode 号、大小、权限、链接数、各种时间戳等。

硬链接与软链接

硬链接

`link("old_name", "new_name")` 创建一个新的目录项指向同一个 inode。一个 inode 可以有多个名字，inode 内部维护引用计数。`unlink("name")` 删除目录项并将引用计数减 1，引用计数归零时文件被真正删除。

所以"删除文件"这个 API 叫 `unlink` 而不是 `delete`。

硬链接的限制：不能跨文件系统（因为 inode 号只在一个文件系统内有意义），不能链接目录（会导致目录树出现环）。

软链接（符号链接）

软链接是一个独立的文件，内容就是目标文件的路径字符串。访问软链接时 OS 自动跟随路径找到目标。

和硬链接的区别：软链接可以跨文件系统，可以链接目录，但目标被删后软链接就成了悬空链接（dangling link）。

mount

`mount` 把一个文件系统挂载到目录树的某个节点上，把多个独立的文件系统统一到一棵树里。

文件系统实现

以 VSFS（Very Simple File System）为模型，理解 ext2/ext3 的核心设计。

磁盘布局

一个文件系统把磁盘分成若干区域：

区域	内容
超级块（superblock）	文件系统元信息：inode 数量、数据块数量、各区域起始位置、魔数等
inode 位图	每个 bit 标记对应 inode 是否被占用
数据位图	每个 bit 标记对应数据块是否被占用
inode 表	存放所有 inode，每个 inode 固定大小（通常 256B）
数据区	存放实际文件数据和目录内容

inode

每个文件一个 inode，包含：

- 文件类型（普通文件、目录、符号链接等）
- 大小、权限、拥有者
- 时间戳（创建、修改、访问）
- 数据块指针

多级索引

如何在 inode 里记录文件数据存在哪些块？

- 直接指针：12 个，每个直接指向一个数据块（4KB × 12 = 48KB）
- 一级间接指针：指向一个块，这个块里全是指针（4KB/4B = 1024 个指针 → 额外 4MB）
- 二级间接指针：指向一个块，里面的指针指向间接块（1024 × 1024 → 额外 4GB）
- 三级间接指针：再套一层（1024^3 → 额外 4TB）

大多数文件很小（平均 2KB 左右），12 个直接指针就够了。只有大文件才需要间接指针。这种不对称设计很聪明——小文件没有间接块的开销。

另一种方案是 `extent`（指针 + 长度）：记录"从块 X 开始连续 N 块"。连续分配的大文件只需要一个 extent。ext4 和 XFS 用 extent。

目录

目录的数据块里存的是条目列表，每个条目包含：

- inode 号
- 条目长度（reclen）
- 文件名长度
- 文件名

删除文件时不会真的移除条目，只是把 reclen 增大（让前一个条目覆盖它）。

空闲空间管理

inode 位图和数据位图各用 1 bit 标记占用/空闲。分配时扫描位图找到空闲位。

ext2/3 等系统会预分配（比如一次分 8 个连续块），让文件数据尽量连续，减少碎片。XFS 用 B 树管理空闲空间。

读文件的路径

读 `/foo/bar` 需要的 I/O 操作：

1. 读根目录 inode（inode 2）
2. 读根目录数据块，找到 `foo` 对应的 inode 号
3. 读 `foo` 的 inode
4. 读 `foo` 的数据块，找到 `bar` 对应的 inode 号
5. 读 `bar` 的 inode
6. 读 `bar` 的数据块

路径每深一层，至少多两次 I/O（读 inode + 读目录数据）。路径越长开销越大。

写文件的路径

写入比读取贵得多。创建一个新文件大约需要 10 次 I/O：

- 读 inode 位图 → 分配 inode → 写 inode 位图
- 写新 inode
- 读目录数据 → 写目录数据（添加新条目）→ 读写目录 inode
- 每写一个数据块：读数据位图 → 写数据位图 → 写数据块 → 更新 inode

缓存

频繁读磁盘太慢。现代系统用统一页缓存（unified page cache），把文件数据和元数据都缓存在内存里——文件系统缓存和虚拟内存系统用同一个缓存池，动态分配。

写操作先写到缓存（`write buffering`），积攒一批后集中写盘（通常延迟 5~30 秒）。好处：

- 批量写入可以调度，减少磁盘寻道
- 有些数据写完很快又被覆盖或删除，直接省掉了磁盘写入
- 降低了单次写操作的感知延迟

代价：断电时缓存中未落盘的数据会丢失。需要 journaling 等机制保证一致性（下一章内容）。

崩溃一致性与日志

文件系统更新一个文件可能涉及多个磁盘写入（数据块、inode、位图）。如果写到一半断电了，磁盘上的数据就不一致了——比如 inode 指向一个没有被位图标记为已用的块，或者位图标记了已用但没有 inode 指向它。

fsck：事后修复

fsck 在重启后扫描整个文件系统，检查各种不一致并修复：

- 超级块是否合理（不合理就用备份）
- 扫描所有 inode，重建空闲位图
- 检查 inode 的引用计数是否和目录中的链接数匹配
- 检查重复块指针、坏块指针
- 检查目录结构合理性（`.` 和 `..` 正确等）

问题：fsck
要扫描整块磁盘。随着磁盘容量增长，可能需要几十分钟甚至几小时。对现代系统来说不可接受。

日志（Journaling）

核心思想：先记日志，再做实际更新。Write-ahead logging，从数据库技术借来的。

数据日志（data journaling）

假设要更新 inode（I）、位图（B）和数据块（D）：

1. 日志写入：把 TxB（事务开始标记）、I、B、D 写入日志区域
2. 日志提交：写入 TxE（事务结束标记）。TxE 必须单独写——因为磁盘只保证 512B 的原子写入，TxE 正好一个扇区
3. 检查点（checkpoint）：把 I、B、D 写到它们在磁盘上的最终位置
4. 释放：标记该事务的日志空间可复用

崩溃恢复时：扫描日志，找到已提交（有 TxE）但未检查点的事务，重放它们。没有 TxE 的事务直接丢弃。

日志是循环缓冲区，有一个日志超级块记录当前头尾位置。

元数据日志（metadata journaling / ordered journaling）

数据日志把数据块也写入日志 → 每个数据写两遍（日志一次，最终位置一次），带宽减半。

元数据日志 只把元数据（inode、位图）写入日志，数据块直接写到最终位置。

关键：数据必须在日志提交之前写完（ordered 模式）。否则崩溃后重放日志，inode 指向的数据块里是旧数据（甚至是别的文件的数据）——泄露信息。

ext3 默认 ordered 模式，NTFS 和 XFS 也用元数据日志。

块复用问题

如果一个目录被删除，它的块被分配给新文件，而旧的日志事务还没检查点——重放日志会用旧目录内容覆盖新文件数据。

解决方案：`revoke record`（撤销记录）。删除操作在日志里写一条撤销记录，恢复时跳过被撤销块的旧事务。

其他方案

`Soft Updates`：精确地排列写入顺序，保证即使崩溃也不会出现危险的不一致。实现极其复杂。

写时复制（Copy-on-Write）：永远不覆盖旧数据，而是写到新位置，最后原子地切换指针。ZFS 和 btrfs 用这种方式，天然不需要日志。

BBC（Backpointer-Based Consistency）：每个块多存一个反向指针指向拥有它的 inode，检查时通过对比正反指针发现不一致。

乐观崩溃一致性：尽可能减少日志写入的磁盘屏障次数，用事务校验和代替一些排序保证，性能更好。